

[MINI SECURE FILE SYSTEM]

Lab Assessment – Challenging Task Report

Subject : Operating Systems Lab
Task Type : Group Activity (Top Students)
Language : Python 3

Submitted by : Abhinav Gomra
Registration no: 24MIP10039
Date : 10 April 2026

> 1. Introduction

A secure file system is a critical component in modern operating systems and networked environments. It ensures that sensitive data is stored, accessed, and managed with appropriate security controls. This project implements a Mini Secure File System in Python that simulates core OS-level concepts including user authentication, file permission models, encrypted storage, and audit logging.

This system demonstrates how an operating system enforces access control policies and protects resources from unauthorized access – a cornerstone of OS security.

> 2. Objectives

- >> Implement a multi-user authentication system with salted password hashing
- >> Provide encrypted file storage using symmetric encryption (Fernet / AES)
- >> Enforce a role-based permission model: owner, read, write, none
- >> Maintain a tamper-evident audit log of all file and user operations
- >> Build an admin dashboard with elevated access for system management
- >> Handle edge cases: unauthorized access, duplicate users, invalid inputs

> 3. System Design

// 3.1 Architecture Overview

The system is structured around three core modules:

- >> Authentication Layer – user registration, login, password hashing
- >> File Storage Engine – create, read, write, delete, encrypt files
- >> Access Control & Log – per-file permissions, audit trail

// 3.2 Class Design

Class	Responsibility	Key Methods
User	User identity & auth	verify_password(), to_dict()
SecureFile	Encrypted file storage	decrypt(), update(), to_dict()

SecureFileSystem	Core FS orchestration	login(), create_file(), grant_permission()
------------------	-----------------------	--

// 3.3 Permission Model

Level	Capabilities
owner	Read, write, delete, grant permissions to others
write	Read and overwrite file contents
read	Read file contents only
none	No access – file is invisible to this user
admin role	Full access to all files and users in the system

> 4. Implementation Details

// 4.1 Authentication & Password Hashing

Passwords are never stored in plain text. SHA-256 is applied over salt+password. Comparison uses hmac.compare_digest to prevent timing attacks:

```
def hash_password(password: str, salt: str) -> str:
    return hashlib.sha256((salt + password).encode()).hexdigest()

def verify_password(self, password: str) -> bool:
    return hmac.compare_digest(
        self.password_hash, hash_password(password, self.salt)
    )
```

// 4.2 File Encryption

File content is encrypted using Fernet (AES-128-CBC + HMAC-SHA256). The key lives in a separate fs.key file:

```
# Encrypting on write:
f = Fernet(key)
self.data = f.encrypt(content)

# Decrypting on read:
return f.decrypt(self.data).decode()
```

// 4.3 Audit Logging

Every action is written to secure_fs.log with timestamp, action type, username, and detail:

```

2025-04-10 14:23:01 | INFO | [LOGIN]      user=alice
2025-04-10 14:23:02 | INFO | [CREATE]    user=alice file=report.txt [encrypted]
2025-04-10 14:23:03 | INFO | [PERMISSION] user=alice granted read to bob
2025-04-10 14:23:04 | INFO | [WRITE_DENIED] user=bob file=report.txt

```

// 4.4 Data Persistence

All state is serialized to `fs_data.json`. Binary encrypted bytes are Base64-encoded before JSON storage. System reloads state on every startup.

> 5. Features & Functionality

Feature	Description
User Registration	New users with unique username and hashed password
Login / Logout	Session-based auth with credential verification
Create File	Stores encrypted file content linked to owner
Read File	Decrypts and returns content if permitted
Write File	Updates file content with re-encryption
Delete File	Owner/admin only deletion with logging
List Files	Shows all files the current user can access
Grant Permission	Owner shares read/write access with other users
Admin Dashboard	System overview: users, files, recent logs
Reset Password	Admin resets any user password with new salt
Delete User	Admin removes a user account from the system

> 6. Security Analysis

// 6.1 Threat Model

```
>> Insider threat – user reading another user's files
```

```
>> Privilege escalation – regular user attempting admin operations
>> Offline attack – attacker reads fs_data.json directly
>> Timing attack – brute-forcing via password comparison timing
```

// 6.2 Mitigations Implemented

Threat	Mitigation
Password cracking	SHA-256 + unique salt per user
Timing attacks	hmac.compare_digest() constant-time comparison
Data exposure (JSON)	All file content AES-encrypted via Fernet
Unauthorized access	Per-file permission checks on every operation
Privilege escalation	Role checks before every admin operation
No audit trail	All actions logged with timestamps

> 7. Test Cases

#	Test Case	Input	Expected Output
1	Register duplicate user	register('alice', ...)	Error: user already exists
2	Wrong password login	login('alice', 'wrong')	Error: invalid credentials
3	Read without permission	bob reads alice private.txt	Error: permission denied
4	Write with read-only perm	bob writes report.txt	Error: permission denied
5	Delete by non-owner	bob deletes alice file	Error: owner/admin only
6	Admin views all users	admin_dashboard()	Full user and file list
7	Encryption check	Read fs_data.json directly	Encrypted bytes not plaintext
8	Grant and verify access	alice grants bob read	bob reads file successfully

> 8. Sample Program Output

Automated demo output from running `secure_fs.py`:

```
Default admin created: admin / admin123
User 'alice' registered.
User 'bob' registered.
File 'report.txt' created (encrypted).
Read back: Q1 Revenue: $1,200,000
bob granted 'read' on 'report.txt'.
LOG: [WRITE_DENIED] user=bob file=report.txt
Permission denied for 'private.txt'.
Password reset for 'bob'.
Demo complete -- check secure_fs.log for audit trail
```

> 9. Challenges & Solutions

- >> Binary data in JSON – Base64-encoded encrypted bytes before serialization
- >> Timing-safe comparison – used `hmac.compare_digest` to prevent side-channel attacks
- >> Graceful degradation – falls back to plain storage if cryptography lib absent
- >> State persistence – `to_dict/from_dict` on all classes for clean serialization
- >> Admin protection – admins bypass per-file maps but still require valid login

> 10. Conclusion

The Mini Secure File System successfully implements core OS security principles in a concise Python codebase. The project demonstrates:

- >> Authentication : salted SHA-256 hashing + timing-safe comparison
- >> Confidentiality : AES-based Fernet encryption of all stored file data
- >> Authorization : per-file permission model + role-based admin access
- >> Accountability : comprehensive audit logging of every operation

The system can be extended with a web frontend, SQLite backend, or network file sharing to further simulate real-world secure file systems.

> References

```
>> Python hashlib docs      -  
    https://docs.python.org/3/library/hashlib.html  
>> Python hmac docs        -  
    https://docs.python.org/3/library/hmac.html  
>> cryptography (Fernet)    - https://cryptography.io/en/latest/fernet/  
>> OWASP Password Storage    - https://cheatsheetseries.owasp.org  
>> Silberschatz et al.      - Operating System Concepts, 10th Edition
```